Chapter 6. Neural Networks and Backpropagation

Basic concepts of neural networks, multilayer perceptrons, backpropagation

Contents

1 From Hand-Designed Features to Learned Functions **2**

2 Neurons, Layers and Shapes **2**

3 Why Nonlinearity and Depth **2**

4 Activation Functions and Outputs **2**

5 Computation Graphs and the Scalar Chain Rule **3**

6 Vector Calculus Rules for Backpropagation **3**

6.1 Backpropagation through an MLP 4

7 Worked Results **4**

7.1 Single neurons: MSE versus cross-entropy (2025 midterm Q2.1) 4

7.2 Residual MLP (2021 midterm Q7) 5

1 From Hand-Designed Features to Learned Functions

A classifier maps an observation $\mathbf{x}$ to an output $y = f(\mathbf{x}; \boldsymbol{\theta})$. Classical pipelines fix features by hand (colour histograms, filter banks) and learn only a final classifier; deep learning trains the representation and the classifier end to end from labelled data. Two properties of features recur: a feature is *invariant* to a transformation if it does not change, and *equivariant* if it changes in the same way (Chapter 9).

2 Neurons, Layers and Shapes

Definition 6.1: multilayer perceptron

A neuron computes $h = \sigma(\mathbf{w}^\top \mathbf{x} + b)$. A fully connected layer stacks neurons:

$$\mathbf{z}^{(\ell)} = W_\ell \mathbf{a}^{(\ell-1)} + \mathbf{b}_\ell, \qquad \mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)}), \qquad \mathbf{a}^{(0)} = \mathbf{x},$$

with $W_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $\mathbf{b}_\ell \in \mathbb{R}^{n_\ell}$, and σ applied elementwise. An MLP is a composition of such layers; a 1000-neuron layer is one layer, not 1000 layers.

Example 6.2: parameter count

A two-layer classifier $\mathbf{s} = W_2 \,\mathrm{ReLU}(W_1 \mathbf{x})$ on CIFAR-10 images ($32 \times 32 \times 3 = 3072$ inputs), hidden width 100, 10 classes: $W_1 \in \mathbb{R}^{100 \times 3072}$, $W_2 \in \mathbb{R}^{10 \times 100}$, so $307{,}200 + 1000 = 308{,}200$ weights (plus 110 biases).

3 Why Nonlinearity and Depth

Without activations layers collapse: $W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2 W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2)$ is still affine, however many layers there are, and an affine score yields linear decision boundaries. Hidden layers are learned embeddings: e.g. $\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$ separates an inner from an outer circle by a plane. A network learns such combinations instead of receiving them by hand; depth composes them hierarchically (strokes $\to$ loops $\to$ digits). Depth increases expressiveness but does not by itself guarantee trainability or generalisation (Chapters 7, 10, 11).

4 Activation Functions and Outputs

Activation	Definition	Derivative
Sigmoid	$\sigma(z) = (1 + e^{-z})^{-1} \in (0, 1)$	$\sigma(1 - \sigma) \in (0, 1/4]$
Tanh	$\tanh z = 2\sigma(2z) - 1 \in (-1, 1)$	$1 - \tanh^2 z \in (0, 1]$
ReLU	$\max(0, z)$	$\mathbf{1}\{z > 0\}$
Leaky ReLU	$\max(az, z)$, $0 < a < 1$	1 or a
ELU	z $(z \geq 0)$, $a(e^z - 1)$ $(z < 0)$	1 or ae^z
Maxout	$\max_k(\mathbf{w}_k^\top \mathbf{x} + b_k)$	routes to the max

Outputs: softmax for mutually exclusive classes (loss $-\log p_y$, score gradient $\mathbf{p} - \mathbf{e}_y$), sigmoid for binary probabilities, identity for regression.

Example 6.3: forward pass

$\mathbf{x} = (1, -2)$, $W_1 = \begin{pmatrix} 1 & 1 \\ 2 & -1 \end{pmatrix}$, $W_2 = \mathbf{I}$, no biases, ReLU hidden layer: $\mathbf{z}_1 = (-1, 4)$, $\mathbf{a}_1 = (0, 4)$, scores $(0, 4)$, softmax $\approx (0.018, 0.982)$.

5 Computation Graphs and the Scalar Chain Rule

Training repeats: zero the gradients, forward pass, loss, backward pass (backpropagation computes derivatives), optimiser update. Backpropagation applies the chain rule on a computation graph: each node multiplies the *upstream* derivative (from later operations) by its *local* derivative.

Example 6.4: $f = (x + y)z$

At $(x, y, z) = (-2, 5, -4)$: $q = x + y = 3$, $f = qz = -12$. Local derivatives $\partial f/\partial q = z = -4$, $\partial f/\partial z = q = 3$, $\partial q/\partial x = \partial q/\partial y = 1$, hence $\partial f/\partial x = \partial f/\partial y = -4$, $\partial f/\partial z = 3$.

Example 6.5: sigmoid neuron

$f = \sigma(w_0x_0 + w_1x_1 + w_2)$ with $w_0 = 2, x_0 = -1, w_1 = -3, x_1 = -2, w_2 = -3$: $z = 1$, $f \approx 0.731$, $\partial f/\partial z = f(1 - f) \approx 0.197$. Then $\partial f/\partial w_0 = 0.197x_0 = -0.197$, $\partial f/\partial x_0 = 0.197w_0 = 0.393$, $\partial f/\partial w_1 = -0.393$, $\partial f/\partial x_1 = -0.590$, $\partial f/\partial w_2 = 0.197$. Decomposing σ into $z \to -z \to e^{-z} \to 1 + e^{-z} \to 1/(\cdot)$ gives the same numbers; treating σ as one node is cheaper.

Remark 6.6: gate patterns

Add gate: distributes the upstream derivative to both inputs. Multiply gate: each input receives upstream $\times$ the other input. Copy (a value used twice): derivatives add. Max gate: routes the derivative to the maximal input. Hence a shared parameter accumulates gradients from all its uses.

6 Vector Calculus Rules for Backpropagation

Gradient convention: $\nabla_{\mathbf{x}} L$ has the shape of $\mathbf{x}$; the Jacobian of $\mathbf{y} \in \mathbb{R}^m$ in $\mathbf{x} \in \mathbb{R}^n$ is $m \times n$ with entries $\partial y_i/\partial x_j$.

Theorem 6.7: rules R1–R6

R1. Sum rule: the Jacobian of a sum is the sum of Jacobians; terms not depending on the variable contribute 0.

R2. Identity: $\partial \mathbf{x}/\partial \mathbf{x} = \mathbf{I}$.

R3. Chain rule: $\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ (outer Jacobian on the left).

R4. Affine map: $\partial(A\mathbf{x} + \mathbf{b})/\partial \mathbf{x} = A$.

R5. Elementwise map: $\partial \sigma(\mathbf{z})/\partial \mathbf{z} = \operatorname{diag}(\sigma'(\mathbf{z}))$, so $\operatorname{diag}(\sigma'(\mathbf{z}))\mathbf{v} = \sigma'(\mathbf{z}) \odot \mathbf{v}$.

R6. Gradient chain rule (scalar L): $\nabla_{\mathbf{x}} L = \left(\frac{\partial \mathbf{u}}{\partial \mathbf{x}}\right)^{\top} \nabla_{\mathbf{u}} L$.

Proof. R6: by R3 the row vector $\partial L/\partial \mathbf{x} = (\partial L/\partial \mathbf{u})(\partial \mathbf{u}/\partial \mathbf{x})$; transposing reverses the order. ■

Theorem 6.8: affine layer

For $\mathbf{z} = W\mathbf{a} + \mathbf{b}$ and $\boldsymbol{\delta} = \nabla_{\mathbf{z}} L$:

$$\nabla_W L = \boldsymbol{\delta}\,\mathbf{a}^\top, \qquad \nabla_{\mathbf{b}} L = \boldsymbol{\delta}, \qquad \nabla_{\mathbf{a}} L = W^\top \boldsymbol{\delta}.$$

For an activation layer $\mathbf{h} = \sigma(\mathbf{z})$: $\nabla_{\mathbf{z}} L = \nabla_{\mathbf{h}} L \odot \sigma'(\mathbf{z})$.

Proof. $\nabla_{\mathbf{a}} L$: R6 with R4. $\nabla_{\mathbf{b}} L$: R6 with $\partial \mathbf{z}/\partial \mathbf{b} = \mathbf{I}$ (R1, R2). $\nabla_W L$: row i of W, $\mathbf{w}_i^\top$, affects L only through $z_i = \mathbf{w}_i^\top \mathbf{a} + b_i$, whose gradient in $\mathbf{w}_i$ is $\mathbf{a}$ (R4); thus $\nabla_{\mathbf{w}_i} L = \delta_i \mathbf{a}$, and stacking rows gives $\boldsymbol{\delta}\mathbf{a}^\top$. Equivalently, componentwise $\partial L/\partial W_{ij} = \delta_i a_j$. Activation: R6 with R5, and a diagonal matrix is symmetric. ■

Remark 6.9: differentials and trace

Matrix gradients can also be read from differentials: if $dL = \operatorname{tr}(K\,dW)$ then $\nabla_W L = K^\top$. For $\mathbf{z} = W\mathbf{a}$, $dL = \boldsymbol{\delta}^\top (dW)\mathbf{a} = \operatorname{tr}(\mathbf{a}\boldsymbol{\delta}^\top dW)$, so $\nabla_W L = \boldsymbol{\delta}\mathbf{a}^\top$.

6.1 Backpropagation through an MLP

With $\boldsymbol{\delta}_\ell = \nabla_{\mathbf{z}^{(\ell)}} L$ (an $n_\ell \times 1$ column), the recursion is

$$\boldsymbol{\delta}_\ell = (W_{\ell+1}^\top \boldsymbol{\delta}_{\ell+1}) \odot \sigma'(\mathbf{z}^{(\ell)}), \qquad \nabla_{W_\ell} L = \boldsymbol{\delta}_\ell\, \mathbf{a}^{(\ell-1)\top}, \qquad \nabla_{\mathbf{b}_\ell} L = \boldsymbol{\delta}_\ell.$$

It follows from applying R6 twice: through the affine map ($W_{\ell+1}^\top$) and through the activation (elementwise product). In a batch, outer products are summed (or averaged) over samples.

Example 6.10: forward example continued backward

Take class 1 as correct: $\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{e}_1 \approx (-0.982, 0.982)$, $\nabla_{W_2} L = \boldsymbol{\delta}_2 \mathbf{a}_1^\top = \begin{pmatrix} 0 & -3.928 \\ 0 & 3.928 \end{pmatrix}$. The first hidden pre-activation was negative, so $\boldsymbol{\delta}_1 = (0, 0.982)$ and $\nabla_{W_1} L = \boldsymbol{\delta}_1 \mathbf{x}^\top = \begin{pmatrix} 0 & 0 \\ 0.982 & -1.964 \end{pmatrix}$: the inactive unit receives no gradient (a "dead ReLU" for this input). For elementwise ReLU, upstream $(4, -1, 5, 9)$ at $\mathbf{x} = (1, -2, 3, -1)$ becomes $(4, 0, 5, 0)$ with no Jacobian ever formed.

7 Worked Results

7.1 Single neurons: MSE versus cross-entropy (2025 midterm Q2.1)

Theorem 6.11: gradient bounds

For $y = \sigma(wx + b)$ and $C = \frac{1}{2}(y - z)^2$: $\nabla C = y(1-y)(y-z)(x, 1)$ and $\|\nabla C\| < \frac{1}{4}\sqrt{1 + x^2}\,(1 + |z|)$. For cross-entropy $C = -[z \log y + (1 - z)\log(1 - y)]$: $\nabla C = (y - z)(x, 1)$.

Proof. Chain rule with $\sigma' = y(1 - y) \le 1/4$ and $|y - z| \le |y| + |z| < 1 + |z|$ since $0 < y < 1$. For cross-entropy, $\partial C/\partial t = -[z(1 - y) - (1 - z)y] = y - z$: the factor σ' cancels. ■

For a saturated wrong neuron ($z = 1$, $wx + b = -10$), the MSE gradient carries the factor $\sigma' \approx 4.5 \times 10^{-5}$ while the cross-entropy gradient is of order one: MSE with sigmoid suffers a learning slowdown. GD reads $(w_{n+1}, b_{n+1}) = (w_n, b_n) - \eta \nabla C$.

7.2 Residual MLP (2021 midterm Q7)

Consider $y_\ell = \sigma(A_\ell y_{\ell-1} + b_\ell) + y_{\ell-1}$ for $2 \le \ell \le L-1$, $y_1 = \sigma(A_1 x + b_1)$, $y_L = A_L y_{L-1} + b_L$ (scalar). Put $z_\ell = A_\ell y_{\ell-1} + b_\ell$, $D_\ell = \operatorname{diag}(\sigma'(z_\ell))$ and $g_\ell = \partial y_L / \partial y_\ell \in \mathbb{R}^{1\times m}$.

Theorem 6.12: residual Jacobians and gradients

$$\frac{\partial y_\ell}{\partial y_{\ell-1}} = I + D_\ell A_\ell \ (2 \le \ell \le L-1), \qquad \frac{\partial y_L}{\partial y_{L-1}} = A_L, \qquad g_\ell = A_L \prod_{j=L-1}^{\ell+1} (I + D_j A_j),$$

$$\frac{\partial y_L}{\partial b_\ell} = g_\ell D_\ell, \qquad \frac{\partial y_L}{\partial A_\ell} = D_\ell g_\ell^\top y_{\ell-1}^\top \quad \Big(\tfrac{\partial y_L}{\partial (A_\ell)_{ij}} = (g_\ell)_i \sigma'((z_\ell)_i)(y_{\ell-1})_j\Big),$$

and $\partial y_L / \partial b_L = 1$, $\partial y_L / \partial A_L = y_{L-1}^\top$.

Proof. R1 splits y_ℓ into $\sigma(z_\ell)$ and $y_{\ell-1}$; R2 gives I; R3 with R5, R4 gives $D_\ell A_\ell$. For b_ℓ, the residual term does not depend on b_ℓ and $\partial z_\ell / \partial b_\ell = I$, so $\partial y_\ell / \partial b_\ell = D_\ell$; chain with g_ℓ. For A_ℓ, the ith row affects y_L only through $(z_\ell)_i$, giving the componentwise formula; it factorises into an i-part and a j-part, i.e. an outer product. ■

If $A_j = 0$ or $\sigma'(z_j) = 0$ for some intermediate j, the factor is $I + 0 = I$, not 0: the identity path carries the gradient, so lower-layer gradients need not vanish. Without the residual, the factor is $D_j A_j = 0$ and all lower gradients are exactly zero. Expanding $\prod(I + D_j A_j) = I + \sum D_j A_j + \cdots$ exhibits the skip path explicitly (Chapter 10).

Chapter 7. Training Deep Networks: SGD, Initialisation, Normalisation

Mini-batch SGD, activations, weight initialisation, learning rates, batch normalisation

Contents

1 Full, Stochastic and Mini-Batch Gradients **2**

2 Activation Functions Revisited **2**

3 Weight Initialisation **2**

3.1 Why the scale matters . . . 2

3.2 Zero and symmetric initialisation . . . 3

3.3 Xavier and He initialisation . . . 3

3.4 Variance of a sigmoid output (2025 midterm Q2.2) . . . 4

4 Learning-Rate Schedules **4**

5 Batch Normalisation **5**

5.1 Training-time computation . . . 5

5.2 Test time . . . 5

6 Dropout **6**

1 Full, Stochastic and Mini-Batch Gradients

For $f(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} f_i(\boldsymbol{\theta})$, the full gradient costs N per-sample gradients. Choosing I uniformly, $\mathbf{g} = \nabla f_I$ is **unbiased**: $\mathbb{E}[\mathbf{g} \mid \boldsymbol{\theta}] = \frac{1}{N}\sum_i \nabla f_i = \nabla f$. A mini-batch averages b such gradients.

Theorem 7.1: mini-batch variance

If the b indices are drawn independently with replacement and a single draw has conditional covariance Σ, the mini-batch gradient has covariance Σ/b. Without replacement it is $\frac{N-b}{b(N-1)}\Sigma$ (with Σ normalised by N).

Proof. With replacement the b terms are iid with covariance Σ; the covariance of their average is $\frac{1}{b^2}\cdot b\Sigma$. Without replacement the pairwise covariances are $-\Sigma/(N-1)$, which gives the finite-population correction. ■

Example 7.2: unbiased but noisy

$f_i(w) = \frac{1}{2}(w - y_i)^2$ with $\mathbf{y} = (1, 3, 5)$ at $w = 0$: per-sample gradients $-1, -3, -5$, full gradient -3. A single random gradient averages to -3; a two-sample batch without replacement gives $-2, -3, -4$, again averaging to -3.

One *iteration* is one parameter update; one *epoch* is one pass through the data (1000 samples, batch 100: 10 iterations per epoch). Full batch: deterministic but expensive; single sample: cheap but noisy; mini-batch: a compromise.

2 Activation Functions Revisited

Sigmoid. Saturates: $\sigma' \le 1/4$ and $\sigma' \to 0$ for large $|z|$. Outputs are positive, so for the next layer $h = W\mathbf{z}$ (with $\mathbf{z}$ the positive sigmoid outputs) $\partial L/\partial W_{ij} = \delta_i z_j$ has the sign of δ_i for every j: all weights into a neuron move in the same direction in one step, producing zig-zag paths. **Tanh.** Zero-centred and odd, $\tanh' \in (0, 1]$, but still saturates near ± 1. **ReLU.** Cheap, derivative 1 on the positive side (no saturation there), but units with negative input get zero gradient (dead ReLU) and outputs are not centred. Leaky ReLU, ELU and maxout keep a nonzero slope or add flexibility.

3 Weight Initialisation

3.1 Why the scale matters

`W = s * np.random.randn(Din, Dout)` draws iid $N(0, s^2)$ weights. For $z_j = \sum_{i=1}^{D_{\text{in}}} W_{ji}x_i$ with zero-mean independent weights of variance σ_W^2, independent of inputs with $\mathbb{E}x_i^2 = q$,

$$\mathbb{E}z_j = 0, \qquad \operatorname{Var} z_j = D_{\text{in}}\sigma_W^2 q$$

(cross terms vanish because $\mathbb{E}[W_{ji}W_{jk}] = 0$ for $i \neq k$). For six tanh layers of width 4096:

- $s = 0.01$: $D_{\text{in}}s^2 = 0.41$; activations shrink towards 0 layer by layer, and weight gradients $\propto$ inputs become tiny.
- $s = 0.05$: $D_{\text{in}}s^2 = 10.24$; pre-activations have standard deviation ≈ 3, tanh saturates (histograms pile up at ± 1), $\tanh' \approx 0$, and the backpropagated signal $\partial L/\partial \mathbf{z} = \sigma'(\cdot)W^\top \partial L/\partial \mathbf{h}$ is almost zero: no learning, although the loss is still large. A zero gradient caused by saturation is not a sign of optimality.

3.2 Zero and symmetric initialisation

Theorem 7.3: zero initialisation freezes a tanh MLP (2022 midterm Q5, 2025 midterm Q3)

For $f(\mathbf{x}) = A_L y_{L-1} + b_L$, $y_\ell = \tanh(A_\ell y_{\ell-1} + b_\ell)$, all parameters initialised to 0 and GD on $\frac{1}{2}\sum_i (f(X_i) - Y_i)^2$: $A_1, \dots, A_L, b_1, \dots, b_{L-1}$ stay 0 forever, and $b_L \to \bar{Y}$ if $0 < \eta < 2/N$.

Proof. Induction on the claim "all parameters except b_L are zero". Forward: $\tanh(0) = 0$ so every hidden output is 0 and $f = b_L$. Backward: $\partial/\partial A_L = \sum_i \delta_L^{(i)} y_{L-1}^\top = 0$; $\boldsymbol{\delta}_{L-1} = (A_L^\top \delta_L) \odot \tanh'(0) = 0$ since $A_L = 0$, and inductively all $\boldsymbol{\delta}_\ell = 0$, so $\partial A_\ell = \boldsymbol{\delta}_\ell y_{\ell-1}^\top = 0$, $\partial b_\ell = 0$. Hence GD does not move them. For b_L: $J = \frac{1}{2}\sum (b_L - Y_i)^2$, $b^{(t+1)} - \bar{Y} = (1 - \eta N)(b^{(t)} - \bar{Y})$. ■

Theorem 7.4: duplicate neurons stay duplicates (2021 midterm Q2)

For $f(x) = \sum_{j=1}^p u_j \sigma(a_j x + b_j)$ trained by Adam (or GD, momentum, RMSProp), if $(a_{p-1}, b_{p-1}, u_{p-1}) = (a_p, b_p, u_p)$ at initialisation, they remain equal for all iterations.

Proof. $\partial f/\partial u_j = \sigma(a_j x + b_j)$, $\partial f/\partial a_j = u_j \sigma'(a_j x + b_j) x$, $\partial f/\partial b_j = u_j \sigma'(a_j x + b_j)$; the loss gradient multiplies these by $\partial \ell/\partial f$, which is shared by all neurons. So equal parameters give equal gradient coordinates. Induct on the statement "the paired coordinates of $\boldsymbol{\theta}^k$, m_1^k, m_2^k agree": it holds at $k = 0$ ($m_1^0 = m_2^0 = 0$), and every Adam operation (EMAs, bias corrections, elementwise division) acts coordinatewise. The optimiser state must be part of the induction hypothesis. ■

Hence identical neurons compute the same function forever: random initialisation is needed to break symmetry.

3.3 Xavier and He initialisation

For an approximately linear activation, preserving the second moment requires $D_{\text{in}} \sigma_W^2 = 1$, i.e. $\sigma_W = 1/\sqrt{D_{\text{in}}}$ (often called "Xavier"; Glorot's compromise $2/(D_{\text{in}} + D_{\text{out}})$ also balances the backward pass).

Theorem 7.5: He initialisation (2022 midterm Q6)

Let $y^+ = A\mathbf{x}$, $\mathbf{x} = \text{ReLU}(\mathbf{y})$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$, where $\mathbf{y}$ is independent of A, its coordinates have mean 0, variance 1, are uncorrelated, and $\mathbf{y} \overset{d}{=} -\mathbf{y}$. Then

(i) $\mathbb{E}x_j^2 = \frac{1}{2}$; (ii) $\mathbb{E}y^+ = 0$; (iii) $\mathbb{E}(y_i^+)^2 = \frac{n_{\text{in}}\sigma_A^2}{2}$;

(iv) $y_1^+, \dots, y_{n_{\text{out}}}^+$ are uncorrelated; (v) $y^+ \overset{d}{=} -y^+$.

Thus $\sigma_A^2 = 2/n_{\text{in}}$ preserves the second moment through ReLU layers.

Proof. (i) $\text{ReLU}(y)^2 = y^2 \mathbf{1}\{y > 0\}$; symmetry makes the positive and negative halves of $\mathbb{E}y^2 = 1$ equal ($y = 0$ contributes nothing). (ii) Independence and $\mathbb{E}A_{ij} = 0$. (iii) $\mathbb{E}(y_i^+)^2 = \sum_{j,k} \mathbb{E}[A_{ij}A_{ik}]\mathbb{E}[x_j x_k] = \sum_j \sigma_A^2 \mathbb{E}x_j^2$. (iv) For $i \neq k$ all $\mathbb{E}[A_{ij}A_{kl}] = 0$. (v) $(A, \mathbf{y}) \overset{d}{=} (-A, \mathbf{y})$ since A is independent of $\mathbf{y}$ and symmetric; apply $F(A, \mathbf{y}) = A\,\text{ReLU}(\mathbf{y})$. ■

Remark 7.6: refinements

Leaky ReLU with slope α: $\mathbb{E}\phi(y)^2 = \frac{1+\alpha^2}{2}$ and $\sigma_A^2 = \frac{2}{(1+\alpha^2)n_{\text{in}}}$. ReLU output is not centred: for $z \sim N(0, v)$, $\mathbb{E}\,\text{ReLU}(z) = \sqrt{v/(2\pi)}$, $\mathbb{E}\,\text{ReLU}(z)^2 = v/2$, so He preserves the second moment, not the variance. Backward, $\partial\ell/\partial\mathbf{x} = A^\top \partial\ell/\partial\mathbf{y}^+$ has variance $n_{\text{out}}\sigma_A^2$: forward and backward preservation ask for n_{in} versus n_{out}.

Theorem 7.7: sigmoid layer (2021 midterm Q4)

For $\mathbf{y} = \sigma(A\mathbf{x})$ with $\sigma(t) \approx \frac{1}{2} + \frac{t}{4}$, $\sigma' \approx \frac{1}{4}$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$ independent of $\mathbf{x}$, and x_j uncorrelated with mean $\frac{1}{2}$ and variance 1: $\mathbb{E}y_i = \frac{1}{2}$, $\operatorname{Var} y_i = \frac{5n_{\text{in}}\sigma_A^2}{64}$. If the upstream gradients $\partial\ell/\partial y_i$ have mean 0, variance 1, are uncorrelated and independent of A, then $\partial\ell/\partial x_j \approx \frac{1}{4}\sum_i A_{ij}\partial\ell/\partial y_i$ has mean 0 and variance $\frac{n_{\text{out}}\sigma_A^2}{16}$.

Proof. $\operatorname{Var}(A_{ij}x_j) = \mathbb{E}A_{ij}^2\mathbb{E}x_j^2 = \sigma_A^2(1 + \frac{1}{4})$ and the terms are uncorrelated; multiply by $\frac{1}{16}$. The backward computation is identical with $\mathbb{E}(\partial\ell/\partial y_i)^2 = 1$. ■

Theorem 7.8: convolution layer (2024 midterm Q5)

For a 3×3, padding-1 convolution $Y_{cij} = \sum_{p,q,r} w_{cpqr}\tilde{X}_{p,i+q-2,j+r-2} + b_c$ with zero-mean unit-variance uncorrelated inputs, $w \overset{\text{iid}}{\sim} N(0, \sigma_w^2)$, $b \overset{\text{iid}}{\sim} N(0, \sigma_b^2)$ independent: $\mathbb{E}Y_{cij} = 0$ everywhere, and $\mathbb{E}Y_{cij}^2 = \sigma_w^2 \cdot \#\{\text{in-range inputs}\} + \sigma_b^2$, i.e. $9C_{\text{in}}\sigma_w^2 + \sigma_b^2$ in the interior, $6C_{\text{in}}\sigma_w^2 + \sigma_b^2$ on edges and $4C_{\text{in}}\sigma_w^2 + \sigma_b^2$ at corners.

Proof. Expand the square: weight–weight terms leave $\sigma_w^2\mathbb{E}\tilde{X}_a^2$ on the diagonal, weight–bias terms vanish, and $\mathbb{E}b_c^2 = \sigma_b^2$. Padded entries are zero. ■

3.4 Variance of a sigmoid output (2025 midterm Q2.2)

For smooth f and X with mean μ, Taylor's theorem $f(X) = f(\mu) + f'(\mu)(X-\mu) + \frac{1}{2}f''(\xi)(X-\mu)^2$ gives $\operatorname{Var} f(X) = f'(\mu)^2 \operatorname{Var} X + \text{Error}$, the error collecting terms with f''. For $X \sim N(0,1)$ and $Y = \sigma(wX + b)$, $\operatorname{Var} Y \approx w^2\sigma'(b)^2$: small weights or large $|b|$ (saturation) give small output variance. An exact bound also holds: if f is L-Lipschitz, $\operatorname{Var} f(X) \le \mathbb{E}(f(X) - f(\mu))^2 \le L^2 \operatorname{Var} X$, so $\operatorname{Var} Y \le w^2/16$.

4 Learning-Rate Schedules

The learning rate controls the update size: too large diverges, too small is slow, and a large early rate followed by a smaller late rate refines the solution and reduces gradient noise.

- Step: multiply by 0.1 at fixed epochs (ResNet: 30, 60, 90), giving $\eta_0, 0.1\eta_0, 0.01\eta_0, 0.001\eta_0$.
- Cosine: $\eta_t = \frac{\eta_0}{2}[1 + \cos(\pi t/T)]$; linear: $\eta_t = \eta_0(1 - t/T)$; inverse square root: $\eta_t = \eta_0/\sqrt{t}$.
- Linear warmup: increase from 0 to $\eta_{\max}$ over the first iterations (e.g. 5000), then decay.

5 Batch Normalisation

5.1 Training-time computation

For a mini-batch $X \in \mathbb{R}^{B\times D}$ (rows: samples, columns: features),

$$\mu_j = \frac{1}{B}\sum_i X_{ij}, \quad v_j = \frac{1}{B}\sum_i (X_{ij}-\mu_j)^2, \quad \hat{X}_{ij} = \frac{X_{ij}-\mu_j}{\sqrt{v_j+\epsilon}}, \quad Y_{ij} = \gamma_j \hat{X}_{ij} + \beta_j.$$

γ, β are learned, so the network can choose the output scale and centre (with $\gamma_j = \sqrt{v_j+\epsilon}$, $\beta_j = \mu_j$ it recovers the input for that batch).

Example 7.9: two features

$X = \begin{pmatrix} 1 & 10 \\ 3 & 14 \end{pmatrix}$: $\mu = (2, 12)$, $v = (1, 4)$, $\hat{X} = \begin{pmatrix} -1 & -1 \\ 1 & 1 \end{pmatrix}$ (ϵ ignored). With $\gamma = (2, 3)$, $\beta = (-1, 5)$, $Y = \begin{pmatrix} -3 & 2 \\ 1 & 8 \end{pmatrix}$.

Theorem 7.10: invariances

With $\epsilon = 0$: (i) each column of $\hat{X}$ has mean 0 and mean square 1; (ii) for $z = \mathbf{w}^\top \mathbf{x} + b$ the BN output does not depend on b and is invariant under $\mathbf{w} \mapsto c\mathbf{w}$, $c > 0$; (iii) consequently $\langle \nabla_{\mathbf{w}} L, \mathbf{w} \rangle = 0$.

Proof. (ii) $z - \mu = \mathbf{w}^\top(\mathbf{x} - \bar{\mathbf{x}})$ removes b; scaling $\mathbf{w}$ scales both $z-\mu$ and $\sqrt{v}$ by c. (iii) $h(c) = L(c\mathbf{w})$ is constant, so $0 = h'(1) = \nabla L(\mathbf{w})^\top \mathbf{w}$. ∎

5.2 Test time

At inference the batch may have size one, and the output must not depend on the other samples. During training BN keeps running averages

$$\mu^{\text{run}} \leftarrow m\,\mu^{\text{run}} + (1-m)\mu_{\text{batch}}, \qquad (\sigma^2)^{\text{run}} \leftarrow m\,(\sigma^2)^{\text{run}} + (1-m)\sigma^2_{\text{batch}}$$

(an exponential moving average, as in Adam; PyTorch's `momentum` argument is $1-m$), and at test time uses

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j^{\text{run}}}{\sqrt{(\sigma_j^2)^{\text{run}} + \epsilon}}, \qquad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j.$$

With a batch of one and batch statistics, $\mu = x$ and $v = 0$ would give $\hat{x} = 0$: all information lost. In code: `model.train()` versus `model.eval()`.

Theorem 7.11: fusion at inference

At test time BN is the affine map $z \mapsto az + c$ with $a_j = \gamma_j / \sqrt{v_j^{\text{run}} + \epsilon}$, $c_j = \beta_j - a_j \mu_j^{\text{run}}$, so $\text{BN}(W\mathbf{x} + \mathbf{b}) = \text{diag}(\mathbf{a})W\mathbf{x} + (\text{diag}(\mathbf{a})\mathbf{b} + \mathbf{c})$ can be folded into the preceding layer.

Remark 7.12: why BN helps

Reported benefits: easier optimisation, better gradient flow, larger learning rates, less sensitivity to initialisation, mild regularisation through batch noise. The original "internal covariate shift" explanation has been challenged; a smoother loss landscape is an alternative

view. In training mode BN couples samples in the backward pass: with $g_i = \partial L/\partial y_i$ and $r = \sqrt{v+\epsilon}$, $\partial L/\partial x_i = \frac{\gamma}{r}(g_i - \bar{g} - \hat{x}_i\,\overline{g\hat{x}})$.

6 Dropout

Inverted dropout multiplies each unit by $m_i/(1-p)$ with $m_i \sim \text{Bernoulli}(1-p)$ during training (so $\mathbb{E}$ = identity) and does nothing at evaluation. It prevents co-adaptation and acts like training an ensemble of sub-networks (AlexNet applies it to the FC layers).

Theorem 7.13: dropout and activations commute iff positively homogeneous (2021 midterm Q3)

With the same mask, $\phi(H \odot \mathbf{z}) = H \odot \phi(\mathbf{z})$ for all masks iff $\phi(0) = 0$ and $\phi(cz) = c\phi(z)$ for $c = 1/(1-p)$. This holds for ReLU and LeakyReLU, but fails for the sigmoid.

Proof. Coordinatewise, a dropped unit gives $\phi(0)$ versus 0, a kept unit $\phi(cz)$ versus $c\phi(z)$. For $c > 0$, ReLU and LeakyReLU preserve sign cases and scale linearly. For the sigmoid with $p = 0.4$ and $z = 0$: $\sigma(0)/0.6 \approx 0.833 \neq \sigma(0) = 0.5$. ■

Chapter 8. Optimisation: Convergence, Momentum and Adaptive Methods

Taylor expansion and the descent lemma, SGD, momentum, Nesterov, AdaGrad, RMSProp, Adam

Contents

1 Gradient Descent and Taylor's Theorem **2**

2 Smoothness and the Descent Lemma **2**

3 Stochastic Gradient Descent **2**

4 Momentum **3**

4.1 Heavy-ball momentum 3

4.2 Nesterov momentum 3

5 Adaptive Methods **4**

5.1 Exponential moving averages and bias correction (2024 midterm Q1) 5

5.2 Special cases and invariances (2022 midterm Q1) 5

5.3 Weight decay is not ℓ_2 regularisation (2024 midterm Q7) 5

1 Gradient Descent and Taylor's Theorem

We minimise $f : \mathbb{R}^d \to \mathbb{R}$; the iterate $\mathbf{x}_t$ is the whole parameter vector. GD is $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$, and $-\nabla f$ is the steepest-descent direction since $\nabla f^\top \mathbf{v} \geq -\|\nabla f\|$ for unit $\mathbf{v}$ (Cauchy–Schwarz).

Theorem 8.1: Taylor along a segment

With $\mathbf{d} = \mathbf{y} - \mathbf{x}$ and $h(t) = f(\mathbf{x} + t\mathbf{d})$,

$$f(\mathbf{y}) - f(\mathbf{x}) = \int_0^1 \nabla f(\mathbf{x}+t\mathbf{d})^\top \mathbf{d}\, dt, \qquad f(\mathbf{y}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{d} + \int_0^1 (1-t)\, \mathbf{d}^\top \nabla^2 f(\mathbf{x}+t\mathbf{d})\mathbf{d}\, dt.$$

Proof. The first is $h(1) - h(0) = \int_0^1 h'$. For the second, $h(1) = h(0) + h'(0) + \int_0^1 \int_0^t h''(s)\, ds\, dt$ and exchanging the order over $0 \leq s \leq t \leq 1$ gives $\int_0^1 (1-s) h''(s)\, ds$. ■

2 Smoothness and the Descent Lemma

Definition 8.2: L-smoothness

f is L-smooth if $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$. For $f \in C^2$ this is equivalent to $-L\mathbf{I} \preceq \nabla^2 f \preceq L\mathbf{I}$ (all Hessian eigenvalues in $[-L, L]$).

Theorem 8.3: descent lemma

If f is L-smooth, $f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2}\|\mathbf{y} - \mathbf{x}\|^2$.

Proof. Subtract $\nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 \nabla f(\mathbf{x})^\top \mathbf{d}\, dt$ from the first Taylor formula, then use Cauchy–Schwarz and smoothness:

$$f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 (\nabla f(\mathbf{x} + t\mathbf{d}) - \nabla f(\mathbf{x}))^\top \mathbf{d}\, dt \leq \int_0^1 Lt\|\mathbf{d}\|^2 dt = \frac{L}{2}\|\mathbf{d}\|^2.$$

■

Theorem 8.4: GD on smooth functions

GD satisfies $f(\mathbf{x}_{t+1}) \leq f(\mathbf{x}_t) - \eta(1 - \frac{L\eta}{2})\|\nabla f(\mathbf{x}_t)\|^2$, so it decreases f when $0 < \eta < 2/L$. With $\eta = 1/L$ and $f^\star = \inf f > -\infty$,

$$\min_{0 \leq t < T} \|\nabla f(\mathbf{x}_t)\|^2 \leq \frac{1}{T} \sum_{t<T} \|\nabla f(\mathbf{x}_t)\|^2 \leq \frac{2L\,(f(\mathbf{x}_0) - f^\star)}{T}.$$

Proof. Put $\mathbf{y} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$ in the descent lemma. With $\eta = 1/L$ the coefficient is $1/(2L)$; summing over t telescopes the left side to $f(\mathbf{x}_0) - f(\mathbf{x}_T) \leq f(\mathbf{x}_0) - f^\star$. ■

This is a stationarity guarantee: for nonconvex f a small gradient may sit at a saddle point or a local minimum. For $f(x) = x^2/2$ ($L = 1$), $x_{t+1} = (1-\eta)x_t$ converges iff $0 < \eta < 2$; for $f = \frac{1}{2}\mathbf{x}^\top H \mathbf{x}$ each eigen-direction contracts by $1 - \eta\lambda_i$, so a small eigenvalue (poor conditioning) means slow progress while $\eta < 2/\lambda_{\max}$ is required for stability.

3 Stochastic Gradient Descent

SGD uses $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \mathbf{g}_t$ with $\mathbb{E}_t \mathbf{g}_t = \nabla f(\mathbf{x}_t)$, where $\mathbb{E}_t$ conditions on the history up to $\mathbf{x}_t$.

Theorem 8.5: second moment is not variance

$\mathbb{E}_t\|\mathbf{g}_t\|^2 = \|\nabla f(\mathbf{x}_t)\|^2 + \mathbb{E}_t\|\mathbf{g}_t - \nabla f(\mathbf{x}_t)\|^2.$

Proof. Expand $\mathbf{g}_t = \nabla f + (\mathbf{g}_t - \nabla f)$; the cross term has conditional mean zero. ■

Theorem 8.6: $T^{-1/2}$ rate for SGD

If f is L-smooth, $\mathbb{E}_t\|\mathbf{g}_t\|^2 \le G$ and $\Delta = f(\mathbf{x}_0) - f^\star$, then for a constant step

$$\frac{1}{T}\sum_{t<T}\mathbb{E}\|\nabla f(\mathbf{x}_t)\|^2 \le \frac{\Delta}{\eta T} + \frac{L\eta G}{2}, \qquad \text{and with } \eta = \sqrt{\tfrac{2\Delta}{LGT}}: \quad \le \sqrt{\frac{2L\Delta G}{T}}.$$

Proof. Descent lemma: $f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\nabla f(\mathbf{x}_t)^\top\mathbf{g}_t + \frac{L\eta^2}{2}\|\mathbf{g}_t\|^2$. Conditional expectation gives $\mathbb{E}_t f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\|\nabla f(\mathbf{x}_t)\|^2 + \frac{L\eta^2}{2}G$. Take total expectation (tower property), sum over $t < T$ (telescoping), use $\mathbb{E}f(\mathbf{x}_T) \ge f^\star$ and divide by ηT. Finally $\frac{a}{\eta} + b\eta$ is minimised at $\eta = \sqrt{a/b}$ with value $2\sqrt{ab}$. ■

The noise term forces $\eta \sim T^{-1/2}$, slower than GD's T^{-1}. Since mini-batches of size B divide the noise variance by B, larger batches reduce G and permit larger steps.

4 Momentum

4.1 Heavy-ball momentum

Definition 8.7: momentum

$\mathbf{v}_{t+1} = \beta\mathbf{v}_t - \eta\nabla f(\mathbf{x}_t)$, $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}$, $\mathbf{v}_0 = \mathbf{0}$, $0 \le \beta < 1$.

Unrolling, $\mathbf{v}_{t+1} = -\eta\sum_{k=0}^{t}\beta^{t-k}\nabla f(\mathbf{x}_k)$: an exponentially weighted sum of past gradients (a velocity). For a constant gradient $\mathbf{g}$, $\|\mathbf{v}_t\| \to \eta\|\mathbf{g}\|/(1-\beta)$, an effective step $1/(1-\beta)$ times larger ($10\times$ for $\beta = 0.9$). Persistent directions accumulate; oscillating directions cancel, which reduces zig-zagging in narrow valleys. Physically the gradient acts as a force and $1-\beta$ as friction: in the buffer form $\mathbf{m}_{t+1} = \mathbf{g}_t + \beta\mathbf{m}_t$ one has $\mathbf{m}_{t+1} - \mathbf{m}_t = \mathbf{g}_t - (1-\beta)\mathbf{m}_t$.

Theorem 8.8: momentum on a quadratic

For $f(x) = \frac{h}{2}x^2$, $x_{t+1} = (1+\beta-\eta h)x_t - \beta x_{t-1}$; the iteration converges iff both roots of $r^2 - (1+\beta-\eta h)r + \beta = 0$ lie in the open unit disk, which for $0 \le \beta < 1$ holds iff $0 < \eta h < 2(1+\beta)$. Complex roots (modulus $\sqrt{\beta}$) mean damped oscillation.

Proof. Substitute $\mathbf{v}_{t+1} = x_{t+1} - x_t$ and $\mathbf{v}_t = x_t - x_{t-1}$ into the velocity update. For the root condition use the Jury/Schur criterion for $r^2 + a_1 r + a_0$: $|a_0| < 1$, $1 + a_1 + a_0 > 0$, $1 - a_1 + a_0 > 0$, which here read $\beta < 1$, $\eta h > 0$, $\eta h < 2(1+\beta)$. ■

4.2 Nesterov momentum

Nesterov evaluates the gradient at the look-ahead point:

$$\mathbf{y}_t = \mathbf{x}_t + \beta\mathbf{v}_t, \qquad \mathbf{v}_{t+1} = \beta\mathbf{v}_t - \eta\nabla f(\mathbf{y}_t), \qquad \mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}.$$

Example 8.9: a numerical illustration

$f = x^2/2$, $x_0 = 1$, $\eta = 0.1$, $\beta = 0.9$. Both methods give $v_1 = -0.1$, $x_1 = 0.9$. Momentum: $v_2 = 0.9(-0.1) - 0.1(0.9) = -0.18$, $x_2 = 0.72$. Nesterov: $y_1 = 0.81$, $v_2 = -0.09 - 0.081 = -0.171$, $x_2 = 0.729$: it measures the slope where it is about to go and brakes earlier (not necessarily closer to the optimum at every step).

Theorem 8.10: two forms of Nesterov (2021 midterm Q6)

Form I: $\psi^{k+1} = \theta^k - \alpha g^k$, $\theta^{k+1} = \psi^{k+1} + \beta(\psi^{k+1} - \psi^k)$, $\psi^0 = \theta^0$. Form II (PyTorch `nesterov=True`): $m^{k+1} = g^k + \beta m^k$, $v^{k+1} = \beta m^{k+1} + g^k$, $\theta^{k+1} = \theta^k - \alpha v^{k+1}$, $m^0 = 0$. Here g^k is evaluated at θ^k. For the same θ^0 and $g^0, g^1, \ldots$ both produce the same $\theta^1, \theta^2, \ldots$

Proof. *Both satisfy one recursion.* Form I with $\psi^{k+1} = \theta^k - \alpha g^k$ and $\psi^k = \theta^{k-1} - \alpha g^{k-1}$ gives, for $k \geq 1$,

$$\theta^{k+1} = \theta^k - \alpha g^k + \beta(\theta^k - \theta^{k-1}) - \alpha\beta(g^k - g^{k-1}).$$

Form II: $\theta^{k+1} = \theta^k - \alpha g^k - \beta(\alpha g^k + \alpha\beta m^k)$, and from the previous step $\alpha\beta m^k = \alpha v^k - \alpha g^{k-1} = \theta^{k-1} - \theta^k - \alpha g^{k-1}$; substituting gives the same recursion. Both start with $\theta^1 = \theta^0 - \alpha(1+\beta)g^0$. A second-order recursion with the same two initial values has the same solution (induction). *Alternative.* Define $\psi^k := \theta^k + \alpha\beta m^k$ in Form II. Then $\psi^0 = \theta^0$, $\psi^{k+1} = \theta^k - \alpha g^k$ and $\psi^{k+1} - \psi^k = -\alpha m^{k+1}$, from which Form I's second equation follows. ■

Interpretation: ψ is the actual position (often written x_t), θ the look-ahead point $x_t + \beta v_t$ at which gradients are taken and which PyTorch stores as the parameter; m is the velocity of ψ scaled by $-1/\alpha$. Both converge to the same limit since $\theta^k - \psi^k = \beta(\psi^k - \psi^{k-1}) \to 0$.

5 Adaptive Methods

All operations below are coordinatewise.

Definition 8.11: AdaGrad, RMSProp, Adam

AdaGrad: $\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$, $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\, \mathbf{g}_t \oslash (\sqrt{\mathbf{r}_t} + \epsilon)$.
RMSProp: $\mathbf{r}_t = \rho \mathbf{r}_{t-1} + (1-\rho)\mathbf{g}_t \odot \mathbf{g}_t$, same update.
Adam: $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$, $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t \odot \mathbf{g}_t$, $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$, $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$, $\mathbf{x}_t = \mathbf{x}_{t-1} - \eta\, \hat{\mathbf{m}}_t \oslash (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$. Typical $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$.

Remark 8.12: what each piece does

AdaGrad's accumulator grows forever, so its effective rate $\eta/\sqrt{r_t}$ shrinks (for a constant gradient g the step is $\eta/\sqrt{t}$). RMSProp forgets old gradients; for constant $g = 2$, $\rho = 0.9$, $\eta = 0.1$, $r_1 = 0.4$, $r_2 = 0.76$, steps $0.316, 0.229$, tending to 0.1. Adam = momentum's numerator (direction) + RMSProp's denominator $\sqrt{\mathbf{v}}$ (per-coordinate scale) + bias correction. (Nesterov plus Adam is NAdam.)

5.1 Exponential moving averages and bias correction (2024 midterm Q1)

Theorem 8.13: EMA of iid gradients

If $g_0, g_1, \dots$ are iid with mean μ and variance s^2, and $m_{k+1} = \beta m_k + (1-\beta) g_k$, $m_0 = 0$, then

$$\mathbb{E} m_k = (1-\beta^k)\mu, \qquad \operatorname{Var} m_k = s^2 \frac{1-\beta}{1+\beta}(1-\beta^{2k}),$$

with limits μ and $s^2 \frac{1-\beta}{1+\beta}$. The corrected $\hat{m}_k = m_k/(1-\beta^k)$ is unbiased with variance $s^2 \frac{(1-\beta)(1+\beta^k)}{(1+\beta)(1-\beta^k)}$ (equal to s^2 at $k=1$).

Proof. m_k depends only on $g_0, \dots, g_{k-1}$, so it is independent of g_k. Mean by induction: $\beta(1-\beta^k)\mu + (1-\beta)\mu = (1-\beta^{k+1})\mu$. Variance: $\operatorname{Var} m_{k+1} = \beta^2 \operatorname{Var} m_k + (1-\beta)^2 s^2$; writing $(1-\beta)^2 = \frac{1-\beta}{1+\beta}(1-\beta^2)$, the induction step is $\beta^2(1-\beta^{2k}) + (1-\beta^2) = 1 - \beta^{2k+2}$. Correction: divide by $(1-\beta^k)^2$ and use $1-\beta^{2k} = (1-\beta^k)(1+\beta^k)$. ■

Without correction Adam's first step is $\frac{1-\beta_1}{\sqrt{1-\beta_2}} \operatorname{sign}(g_1)\eta \approx 3.16\,\eta$; with correction $\hat{m}_1 = g_1$, $\hat{v}_1 = g_1^2$ and the step is exactly $\eta \operatorname{sign}(g_1)$. The correction removes the zero-initialisation bias at the price of a larger early variance.

5.2 Special cases and invariances (2022 midterm Q1)

Theorem 8.14: Adam contains RMSProp and signSGD

For Adam without bias correction: $\beta_1 = 0$ gives RMSProp with $\rho = \beta_2$; $\beta_1 = \beta_2 = 0$, $\epsilon = 0$ give signSGD $\theta_{k+1} = \theta_k - \eta \operatorname{sign}(g_k)$ (if no coordinate of g_k is 0). With $\epsilon = 0$, Adam's iterates are unchanged when the loss is multiplied by $c > 0$, while SGD's step scales by c.

Proof. $\beta_1 = 0$ makes $m_{k+1} = g_k$. With $\beta_2 = 0$ also, $\sqrt{v_{k+1}} = |g_k|$. Scaling $g \mapsto cg$ scales m by c and $\sqrt{v}$ by c. ■

5.3 Weight decay is not ℓ_2 regularisation (2024 midterm Q7)

For plain SGD, ℓ_2 regularisation $\theta - \eta(g + \lambda\theta) = (1-\eta\lambda)\theta - \eta g$ is the same as weight decay. With momentum it is not:

Theorem 8.15: no matching exists

SGD with momentum $v_{k+1} = g_k + \beta v_k$, $\theta_{k+1} = \theta_k - \alpha v_{k+1}$ and $\alpha, \beta > 0$. Weight decay: $\theta_{k+1} = \theta_k - \alpha v_{k+1} - \mu\theta_k$. ℓ_2: $v_{k+1} = (g_k + \lambda\theta_k) + \beta v_k$. No choice of $\mu, \lambda > 0$ makes the two methods equivalent.

Proof. Take $g_k \equiv 0$, $v_0 = 0$, $\theta_0 \neq 0$. Weight decay: $v_k = 0$ and $\theta_k = (1-\mu)^k \theta_0$. ℓ_2: $v_1 = \lambda\theta_0$, $\theta_1 = (1-\alpha\lambda)\theta_0$, $v_2 = \lambda(1-\alpha\lambda+\beta)\theta_0$, $\theta_2 = [(1-\alpha\lambda)^2 - \alpha\beta\lambda]\theta_0$. Matching θ_1 forces $\mu = \alpha\lambda$, and then matching θ_2 forces $\alpha\beta\lambda = 0$, a contradiction. (Indeed the ℓ_2 sequence is geometric for no ratio.) ■

In Adam the ℓ_2 gradient $\lambda\theta$ is divided by its own RMS, so every weight moves by about η regardless of its size ($\theta_1 = \theta_0 - \eta \operatorname{sign}(\theta_0)$ when $g \equiv 0$), losing proportional shrinkage; AdamW applies the decay $(1-\eta\lambda)\theta$ outside the adaptive scaling.

Chapter 9. Convolutional Neural Networks

Convolution, stride, padding, pooling, equivariance

Contents

1 Why a New Layer **2**

2 Convolution as a Local Dot Product **2**

2.1 A fixed filter: moving average 2

2.2 Single and multiple channels 2

2.3 Parameter counts 3

3 Stride, Padding and Output Size **3**

4 Learning Filters **4**

5 Shift Equivariance **4**

6 1×1 Convolution, Pooling and Global Average Pooling **4**

7 Algebraic Facts from Past Exams **5**

1 Why a New Layer

An image $X \in \mathbb{R}^{H\times W\times C}$ fed to a fully connected layer is flattened, and $z = A\mathbf{x} + \mathbf{b}$ with $A \in \mathbb{R}^{M\times HWC}$ has $M(HWC+1)$ parameters: for $32\times 32\times 3$ input and $M = 1000$, already 3,073,000. Flattening loses no numbers, but an unrestricted matrix does not encode which pixels are neighbours or that a pattern can appear anywhere. Two assumptions about images motivate convolution:

- **Locality**: nearby pixels jointly form edges, corners and textures, so a unit can look at a small patch (local connectivity).
- **Stationarity**: the same pattern may occur at any position, so the same detector should be used everywhere (weight sharing).

Definition 9.1: equivariance and invariance

Let $T_{\boldsymbol{\tau}}$ shift an image, $(T_{\boldsymbol{\tau}}X)_{i,j,c} = X_{i-\tau_1, j-\tau_2, c}$. A feature extractor F is shift *equivariant* if $F(T_{\boldsymbol{\tau}}X) = T_{\boldsymbol{\tau}}F(X)$ (the output moves with the input) and a classifier G is shift *invariant* if $G(T_{\boldsymbol{\tau}}X) = G(X)$ (the output does not change). More generally, for a group acting by $\rho_{\text{in}}(g)$, $\rho_{\text{out}}(g)$, f is equivariant if $f(\rho_{\text{in}}(g)x) = \rho_{\text{out}}(g)f(x)$; invariance is the case $\rho_{\text{out}} = \text{id}$.

2 Convolution as a Local Dot Product

2.1 A fixed filter: moving average

With the 3×3 averaging kernel (1/9 everywhere), $Y_{i,j} = \frac{1}{9}\sum_{u,v=0}^{2} X_{i+u,j+v}$: on an image of 0s and 90s a window with r bright pixels outputs $10r$. Averaging suppresses isolated pixels but blurs edges; a 5×5 average smooths more. A CNN *learns* the kernel; learned weights may be negative and need not sum to one.

2.2 Single and multiple channels

Definition 9.2: convolutional layer

With D filters $W \in \mathbb{R}^{D\times K\times K\times C}$, biases $\mathbf{b} \in \mathbb{R}^D$, stride S, padding P,

$$Z_{i,j,d} = b_d + \sum_{u=0}^{K-1}\sum_{v=0}^{K-1}\sum_{c=1}^{C} W_{d,u,v,c}\,\tilde{X}_{Si+u,\,Sj+v,\,c},$$

where $\tilde{X}$ is the padded input. Each output is a dot product of a $K\times K\times C$ patch with a filter spanning *all* input channels, plus a bias. The array $Z_{\cdot,\cdot,d}$ is the feature map of filter d; stacking the D maps gives D output channels. This operation is cross-correlation (no kernel flip); since kernels are learned, the distinction does not matter for expressiveness.

Example 9.3: one channel, 2×2 filter

$X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{pmatrix}$, $W = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}$, $b = 1$, $S = 1$, $P = 0$: $Z = \begin{pmatrix} 4 & 8 \\ 1 & 1 \end{pmatrix}$. The same four weights and bias are reused at the four positions.

Example 9.4: a 5×5 example

With a 5×5 image and $W = \begin{pmatrix} 1 & 2 & 3 \\ -4 & 7 & 4 \\ 2 & -5 & 1 \end{pmatrix}$ ($b = 0$): unpadded $Z = \begin{pmatrix} 51 & 66 & 20 \\ 31 & 49 & 101 \\ 15 & 53 & -2 \end{pmatrix}$; with $P = 1$ the output is 5×5 and its central 3×3 block equals the unpadded output (those windows contain the same pixels).

2.3 Parameter counts

One $5 \times 5 \times 3$ filter has 75 weights and one bias; six filters on $32 \times 32 \times 3$ give a $28 \times 28 \times 6$ output with $6(75 + 1) = 456$ parameters. For the same input and output shape:

Layer	Parameters
Fully connected	$4704(3072 + 1) = 14{,}455{,}392$
Locally connected, unshared	$28^2 \cdot 6(75 + 1) = 357{,}504$
Convolution (shared)	$6(75 + 1) = 456$

Locality removes long-range connections; sharing removes independent copies. As a matrix acting on the flattened input, convolution is sparse with repeated entries (a 1D length-2 filter (a, b) on $(x_1, \ldots, x_4)$ gives a banded matrix with only two free numbers).

3 Stride, Padding and Output Size

Theorem 9.5: output size

For input length N, kernel K, stride S, padding P (symmetric), the number of output positions is

$$N' = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1, \qquad C' = D.$$

Proof. The padded length is $N + 2P$. A window starting at tS ends at $tS + K - 1$, which must be $\leq N + 2P - 1$, i.e. $t \leq (N + 2P - K)/S$. The admissible integers are $t = 0, \ldots, \lfloor (N + 2P - K)/S \rfloor$. ■

$N = 7$, $K = 3$: $S = 1 \Rightarrow 5$, $S = 2 \Rightarrow 3$, $S = 3 \Rightarrow 2$ (the last does not "fit" exactly; a border is left unused). For $S = 1$ and odd K, $P = (K - 1)/2$ preserves the size (padding $1, 2, 3$ for $K = 3, 5, 7$). Zero padding is an explicit boundary assumption, not recovered information.

Example 9.6: further examples

Input $52 \times 52 \times 128$, $D = 256$. (1) $K = 3, S = 1, P = 1$: output $52 \times 52 \times 256$, parameters $256(9 \cdot 128 + 1) = 295{,}168$; each filter is $3 \times 3 \times 128$. (2) $K = 4, S = 2, P = 1$: $\lfloor (52 + 2 - 4)/2 \rfloor + 1 = 26$, output $26 \times 26 \times 256$, parameters $256(16 \cdot 128 + 1) = 524{,}544$. Parameters, activations (HWD) and multiply–accumulates ($\approx HWDK^2C$) are different costs.

Stacking: $32 \times 32 \times 3 \to 28 \times 28 \times 6 \to 24 \times 24 \times 10$ with ten $5 \times 5 \times 6$ filters ($10(150 + 1) = 1510$ parameters). Two stacked stride-1 5×5 layers see a 9×9 patch (receptive field $5 + 4$). With strides, $r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}$ and $j_\ell = S_\ell j_{\ell-1}$, starting from $r_0 = j_0 = 1$.

4 Learning Filters

For an unpadded stride-1 layer with upstream $G_{i,j,d} = \partial L/\partial Z_{i,j,d}$,

$$\frac{\partial L}{\partial W_{d,u,v,c}} = \sum_{i,j} G_{i,j,d} X_{i+u,j+v,c}, \qquad \frac{\partial L}{\partial b_d} = \sum_{i,j} G_{i,j,d} :$$

every position contributes to the update of the shared weight (the backward counterpart of weight sharing).

5 Shift Equivariance

Theorem 9.7: convolution commutes with shifts

On an infinite grid (or with circular indices), stride-1 convolution satisfies $C_W(T_{\boldsymbol{\tau}} X) = T_{\boldsymbol{\tau}} C_W(X)$, and pointwise nonlinearities also commute with shifts. Global average pooling of a circularly shifted map is unchanged, so GAP $\circ$ (convolutions, ReLUs) is shift invariant.

Proof. $[C_W(T_{\boldsymbol{\tau}} X)]_{i,j,d} = b_d + \sum_{u,v,c} W_{d,u,v,c} X_{i+u-\tau_1,j+v-\tau_2,c} = [C_W X]_{i-\tau_1,j-\tau_2,d}$; the coefficients do not depend on the absolute position (i, j). A circular shift permutes positions, and an average is invariant under permutations. ■

Remark 9.8: when exactness fails

With zero padding, content shifted across the border is lost and zeros enter, so equality holds only away from the boundary. A stride-S layer (or pooling) is compatible only with shifts that are multiples of S: for width-2, stride-2 max pooling M, $M(T_2 x) = T_1 M(x)$ but $M(T_1 x)$ need not be a shift of $M(x)$ (e.g. $x = (0, 1, 2, 0)$, $M(x) = (1, 2)$, $M(T_1 x) = (0, 2)$).

6 1×1 Convolution, Pooling and Global Average Pooling

1×1 convolution. At each position $\mathbf{y}_{i,j} = A\mathbf{x}_{i,j} + \mathbf{b}$ with $A \in \mathbb{R}^{D \times C}$ shared across positions: it mixes channels, not neighbours. $56 \times 56 \times 64 \to 56 \times 56 \times 32$ costs $32(64 + 1) = 2080$ parameters.

Pooling. Channelwise, no parameters: $Y^{\max}_{i,j,c} = \max_{u,v} X_{iS+u,jS+v,c}$ or the average. For $X = \begin{pmatrix} 5 & 10 & 6 & 7 \\ 4 & 0 & 4 & 3 \\ 2 & 4 & 3 & 4 \\ 0 & 8 & 1 & 6 \end{pmatrix}$ and 2×2, stride 2: average $\begin{pmatrix} 4.75 & 5 \\ 3.5 & 3.5 \end{pmatrix}$, max $\begin{pmatrix} 10 & 7 \\ 8 & 6 \end{pmatrix}$. Pooling reduces resolution and gives limited robustness to small shifts within a window. Backward: max pooling routes the gradient to the maximiser; average pooling spreads g/K^2.

Global average pooling. $g_c = \frac{1}{HW} \sum_{i,j} X_{i,j,c}$ maps $H \times W \times C \to C$; a classifier on top needs $Q(C + 1)$ instead of $Q(HWC + 1)$ parameters. It discards spatial arrangement entirely.

7 Algebraic Facts from Past Exams

Theorem 9.9: positive homogeneity (2024 midterm Q6)

A network built only from bias-free convolutions and linear layers, ReLU, max pooling, reshaping and dropout satisfies $f(\lambda X) = \lambda f(X)$ for all $\lambda \geq 0$.

Proof. Each component is positively homogeneous: linear maps for all λ; $\max(0, \lambda t) = \lambda \max(0, t)$ and $\max_i \lambda x_i = \lambda \max_i x_i$ for $\lambda \geq 0$ (scaling by $\lambda > 0$ preserves order and signs; $\lambda = 0$ is trivial); dropout multiplies by a fixed mask. Compositions of positively homogeneous maps are positively homogeneous: $g_1(g_2(\lambda X)) = g_1(\lambda g_2(X)) = \lambda g_1(g_2(X))$. Biases break the property, e.g. $\mathrm{ReLU}(x+1)$ at $x = 0$. ■

Theorem 9.10: conv–MP–ReLU orders coincide (2022 midterm Q2)

conv→MaxPool→ReLU, conv→ReLU→MaxPool and conv→ReLU→MaxPool→ReLU are the same function.

Proof. For nondecreasing ϕ, $\max_i \phi(x_i) = \phi(\max_i x_i)$: $\phi(x_i) \leq \phi(\max x)$ for all i with equality at the maximiser. ReLU is nondecreasing and elementwise, and max pooling is a windowwise max, so $\mathrm{ReLU} \circ \mathrm{MP} = \mathrm{MP} \circ \mathrm{ReLU}$; finally $\mathrm{ReLU} \circ \mathrm{ReLU} = \mathrm{ReLU}$. (For average pooling only avg ReLU $\geq$ ReLU avg holds, by convexity.) ■

Theorem 9.11: convolutions that implement given maps (2021 midterm Q1, 2022 midterm Q3)

With PyTorch's convention $Y_{o,i,j} = b_o + \sum_c \sum_{k,l=1}^{3} w_{o,c,k,l} \tilde{X}_{c,i+k-2,j+l-2}$ (3×3, padding 1), reading the value at offset $(\Delta i, \Delta j)$ requires a 1 at $(k, l) = (2 + \Delta i, 2 + \Delta j)$.

- Identity $C \to C$: $w_{c,c',k,l} = \delta_{cc'} \delta_{k2} \delta_{l2}$, $\mathbf{b} = \mathbf{0}$.
- Colourwise finite differences $Y_{2c-1} = X_{c,i+1,j} - X_{c,i,j}$, $Y_{2c} = X_{c,i,j+1} - X_{c,i,j}$: $w_{2c-1,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, $w_{2c,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$, all other slices zero; zero padding supplies the out-of-range zeros.

Chapter 10. CNN Architectures, Residual Learning and Saliency

AlexNet, LRN, ZFNet, VGG, GoogLeNet/Inception, ResNet, saliency maps

Contents

1 Vocabulary and Costs **2**

2 AlexNet **2**

3 ZFNet and VGG **2**

4 GoogLeNet and the Inception Module **2**

4.1 Naive module 2

4.2 Bottleneck module 3

4.3 Split–transform–merge (2021 midterm Q5) 3

5 Network in Network: Exact versus Approximate Invariance (2022 midterm Q7) **3**

6 Why Depth Can Fail, and Residual Networks **4**

7 Saliency Maps **4**

1 Vocabulary and Costs

A network maps an image to class scores and then probabilities, $X \mapsto h(X) \mapsto \mathbf{s}(X) \in \mathbb{R}^Q \mapsto \mathbf{p}(X)$. For a convolution with D filters $K \times K \times C$ producing $H' \times W' \times D$:

$$N_{\text{param}} = D(K^2C + 1), \qquad N_{\text{act}} = H'W'D, \qquad N_{\text{MAC}} \approx H'W'DK^2C.$$

An FC layer $\mathbb{R}^n \to \mathbb{R}^m$ has $m(n + 1)$ parameters. Top-1 error asks whether the highest score is correct; top-5 error asks whether the true label is missing from the five highest scores.

2 AlexNet

Five convolutional and three FC layers (eight weight layers):

Stage	Settings	Output	
Input	RGB	$227 \times 227 \times 3$	
Conv1	96, 11×11, $S = 4$, $P = 0$	$55 \times 55 \times 96$	$((227 - 11)/4 + 1)$
Pool1, Norm1	3×3, $S = 2$; LRN	$27 \times 27 \times 96$	
Conv2	256, 5×5, $P = 2$	$27 \times 27 \times 256$	
Pool2, Norm2	3×3, $S = 2$; LRN	$13 \times 13 \times 256$	
Conv3, Conv4	384, 3×3, $P = 1$	$13 \times 13 \times 384$	
Conv5	256, 3×3, $P = 1$	$13 \times 13 \times 256$	
Pool3	3×3, $S = 2$	$6 \times 6 \times 256 = 9216$	
FC6, FC7, FC8	$9216 \to 4096 \to 4096 \to 1000$	1000 scores	

The FC layers dominate: $4096 \cdot 9217 = 37{,}752{,}832$, $4096 \cdot 4097 = 16{,}781{,}312$, $1000 \cdot 4097 = 4{,}097{,}000$, together 58,631,144 parameters, whereas Conv1 has $96(11^2 \cdot 3 + 1) = 34{,}944$. Design choices:

- **ReLU**: avoids sigmoid/tanh saturation on the positive side.
- **Large first stride**: $227 \to 55$ immediately reduces memory and computation (at the cost of detail).
- **Local response normalisation**: $a_{i,j,c} \mapsto a_{i,j,c} / \big(k + \alpha \sum_{r \in N(c)} a_{i,j,r}^2\big)^{\beta}$ normalises across neighbouring channels (not batch normalisation).
- **Dropout** 0.5 in the FC layers against overfitting.

3 ZFNet and VGG

ZFNet tunes AlexNet's kernel sizes and strides. VGG uses deeper stacks of small 3×3 convolutions: two stride-1 3×3 layers have a 5×5 receptive field with $18C^2$ weights (versus $25C^2$), three have 7×7 with $27C^2$ (versus $49C^2$), and interleaved ReLUs add nonlinearity. Receptive fields obey $r_\ell = r_{\ell-1} + (K_\ell - 1)j_{\ell-1}$, $j_\ell = S_\ell j_{\ell-1}$.

4 GoogLeNet and the Inception Module

4.1 Naive module

For input $28 \times 28 \times 256$, parallel branches with padding chosen to keep 28×28: 1×1 (128), 3×3 (192), 5×5 (96), 3×3 max pool (stride 1). Channel concatenation gives $128 + 192 + 96 + 256 = 672$ channels. Weights: $256 \cdot 128 + 9 \cdot 256 \cdot 192 + 25 \cdot 256 \cdot 96 = 1{,}089{,}536$.

4.2 Bottleneck module

Insert 1×1 reductions to 64 channels before the 3×3 and 5×5 convolutions and after pooling: outputs $128 + 192 + 96 + 64 = 480$ channels and

$$32{,}768 + (16{,}384 + 110{,}592) + (16{,}384 + 153{,}600) + 16{,}384 = 346{,}112 \text{ weights}$$

(about 330k), a 68% reduction.

Theorem 10.1: when a bottleneck saves weights

A direct $K \times K$ map $C \to D$ costs K^2CD; reducing to R channels first costs $CR + K^2RD$. This is cheaper iff $R < \dfrac{K^2CD}{C + K^2D}$ (for $K = 5, C = 256, D = 96$ the threshold is ≈ 231).

The 1×1 convolution is a channel-mixing matrix applied at every position (Chapter 9). GoogLeNet stacks such modules (22 weight layers) and replaces the large FC stack by global average pooling plus a final classifier.

4.3 Split–transform–merge (2021 midterm Q5)

A $256 \to 128 \to 128 \to 256$ conv–ReLU block (1×1, 3×3 with padding 1, 1×1, all with biases) has $32{,}896 + 147{,}584 + 33{,}024 = 213{,}504$ parameters. Thirty-two parallel $256 \to 4 \to 4 \to 256$ paths summed at the end have $32(1028 + 148 + 1280) = 78{,}592$ parameters; the 3×3 cost drops by the group factor 32. Implementation: three `nn.ModuleLists` of 32 convolutions, ReLU inside each path, then a sum (equivalently grouped convolutions with `groups=32`).

5 Network in Network: Exact versus Approximate Invariance (2022 midterm Q7)

NiN stacks mlpconv blocks (a spatial convolution followed by 1×1 convolutions, ReLUs, 3×3 stride-2 max pooling with ceil mode, dropout) and ends with average pooling over the final $8 \times 8 \times 10$ map. Take X with a single 1 at $(j, k) = (19, 19)$ and $\tilde{X}$ the same shifted to $(23, 19)$ (0-based, no biases, eval mode). Tracking where outputs can be nonzero: bias-free layers map zeros to zeros; a $K \times K$ convolution with padding $(K-1)/2$ widens the support by $(K-1)/2$; 1×1 convolutions and ReLU keep it; pool output p sees rows $2p, 2p+1, 2p+2$.

	rows for X	rows for $\tilde{X}$
conv1 (5×5)	17–21	21–25
out1 (16×16)	8–10	10–12
conv2 (5×5)	6–12	8–14
out2 (8×8)	2–6	3–7
out3 (3×3 conv)	1–7	2–8, truncated to 2–7

Columns stay 8–10, 2–6, 1–7. The shift 4 is a multiple of the total stride 4, so out1 is exactly shifted by 2 and out2 by 1; then out3 of $\tilde{X}$ equals out3 of X shifted down one row, except that row 7 of X's map would land on row 8, outside the 8×8 map. Hence the averaged outputs differ exactly by $\frac{1}{64}\sum_k \text{out3}(X)_{c,7,k}$: approximately equal, but not exactly equal in general.

6 Why Depth Can Fail, and Residual Networks

Deeper plain networks can have *higher training error*: this is not overfitting but an optimisation (degradation) problem. A deeper network that can represent identity layers contains the shallower one, so its *optimal* training loss is not worse, yet gradient methods may not find it. For $\mathbf{x}_{\ell+1} = f_\ell(\mathbf{x}_\ell)$, $\nabla_{\mathbf{x}_0} L = J_0^\top \cdots J_{L-1}^\top \nabla_{\mathbf{x}_L} L$; if every Jacobian has norm at most $\rho < 1$ the gradient shrinks like ρ^L ($0.9^{50} \approx 0.005$).

Theorem 10.2: residual block

$\mathbf{y} = \mathbf{x} + F(\mathbf{x}; \boldsymbol{\theta})$ gives $\dfrac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + J_F(\mathbf{x})$ and $\nabla_{\mathbf{x}} L = \mathbf{g} + J_F^\top \mathbf{g}$ with $\mathbf{g} = \nabla_{\mathbf{y}} L$.

The identity path carries the gradient even when J_F is small, and the block can represent the identity by $F = 0$: the network learns a residual $F = H - \mathrm{id}$ rather than the whole map H. This is a reparametrisation, not a guarantee (e.g. $J_F = -\mathbf{I}$ cancels the gradient; a ReLU after the sum alters the identity). If shapes differ, use a projection shortcut $\mathbf{y} = P(\mathbf{x}) + F(\mathbf{x})$ (e.g. a 1×1 or strided convolution). Unlike Inception's concatenation, residual addition keeps the channel count. A bottleneck block $256 \to 64 \to 64 \to 256$ (1×1, 3×3, 1×1) has $2 \cdot 256 \cdot 64 + 9 \cdot 64^2 = 69{,}632$ weights versus $18 \cdot 256^2 = 1{,}179{,}648$ for two full-width 3×3 layers. In the CIFAR-10 plots, residual networks remove the training-error degradation seen in plain networks. The full backpropagation formulas for a residual MLP are derived in Chapter 6 (2021 midterm Q7).

7 Saliency Maps

To see which pixels a prediction is sensitive to, differentiate the selected class score with respect to the input (not the weights):

$$G_{i,j,k} = \frac{\partial s_c(X)}{\partial X_{i,j,k}}, \qquad M_{i,j} = \max_k |G_{i,j,k}|.$$

By Taylor's theorem $s_c(X + \Delta X) \approx s_c(X) + \langle \nabla_X s_c(X), \Delta X \rangle$: a large $|G|$ means a small change of that pixel changes the score a lot, and the sign says in which direction. For $s(\mathbf{x}) = 2x_1 - 3x_2 + 0.1x_3$, the gradient $(2, -3, 0.1)$ ranks x_2 first. Saliency is local sensitivity, not a complete causal explanation.

Chapter 11. Universal Approximation and Barron's Theorem

Universal approximation: measures, normed spaces, Cybenko's theorem; the Barron class

Contents

1 Three Sources of Error **2**

2 Preliminaries from Analysis **2**

2.1 Measures 2

2.2 Normed spaces and density 2

3 Cybenko's Universal Approximation Theorem **3**

3.1 The functional-analytic lemmas 3

3.2 Density for discriminatory functions 4

3.3 Continuous sigmoidals are discriminatory 4

4 Barron's Theorem: a Dimension-Free Rate **4**

4.1 Fourier preliminaries 4

4.2 Proof idea: Maurey's empirical method 5

1 Three Sources of Error

Let f^* be the ideal target, $\mathcal{F}$ the class of functions a given architecture can represent, f_{app} the best element of $\mathcal{F}$, f_{gen} the best element on the finite training sample (empirical risk minimiser) and f_{opt} what the optimiser actually returns. The total error splits as

$$\underbrace{f^* \to f_{\text{app}}}_{\text{approximation}} + \underbrace{f_{\text{app}} \to f_{\text{gen}}}_{\text{generalisation (estimation)}} + \underbrace{f_{\text{gen}} \to f_{\text{opt}}}_{\text{optimisation}}.$$

Chapters 7–8 dealt with optimisation; this chapter shows that the approximation error of one-hidden-layer networks can be made arbitrarily small (universality) and, for Barron functions, quantifies it.

2 Preliminaries from Analysis

2.1 Measures

Definition 11.1: measures on a compact set $K \subset \mathbb{R}^n$

The **Borel sets** of K form the smallest collection containing all open subsets of K and closed under complements in K and countable unions. A (nonnegative) **measure** μ assigns $\mu(A) \in [0, \infty]$ to Borel sets with

$$\mu(\emptyset) = 0, \qquad \mu\Big(\bigcup_{j=1}^{\infty} A_j\Big) = \sum_{j=1}^{\infty} \mu(A_j) \quad \text{for pairwise disjoint } A_j.$$

A **signed measure** is the same with real (possibly negative) values, e.g. a difference of two finite measures. $\mu = 0$ means $\mu(A) = 0$ for every Borel set A. $M(I_n)$ denotes the finite signed regular Borel measures on the unit cube $I_n = [0,1]^n$.

2.2 Normed spaces and density

Definition 11.2: linear and normed spaces

A real **linear space** is a set closed under addition and multiplication by reals obeying the usual vector-space rules; e.g. $C(K) = \{f : K \to \mathbb{R} \text{ continuous}\}$, since $af + bg \in C(K)$ with $(af + bg)(x) = af(x) + bg(x)$. A **normed linear space** is a linear space X with $\|\cdot\| : X \to [0, \infty)$ such that

$$\|u\| = 0 \iff u = 0, \qquad \|cu\| = |c|\|u\|, \qquad \|u + v\| \le \|u\| + \|v\|.$$

On $C(K)$ we use the sup norm $\|f\|_\infty = \max_{x \in K} |f(x)|$. A subset $D \subseteq X$ is **dense** in X if for every $f \in X$ and $\varepsilon > 0$ there is $g \in D$ with $\|f - g\| < \varepsilon$.

Density in $(C(I_n), \|\cdot\|_\infty)$ is exactly uniform approximation: every continuous function can be approximated within ε at every point simultaneously.

3 Cybenko's Universal Approximation Theorem

Definition 11.3: sigmoidal and discriminatory

$\sigma : \mathbb{R} \to [0,1]$ is **sigmoidal** if $\lim_{t\to-\infty}\sigma(t) = 0$ and $\lim_{t\to+\infty}\sigma(t) = 1$. For $\mu \in M(I_n)$, σ is **discriminatory** if

$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu(\mathbf{x}) = 0 \quad \text{for all } \mathbf{w} \in \mathbb{R}^n,\ \theta \in \mathbb{R} \qquad \Longrightarrow \qquad \mu = 0.$$

Theorem 11.4: Cybenko 1989 (UAT)

Let σ be any continuous sigmoidal function. Then finite sums

$$G(\mathbf{x}) = \sum_{j=1}^{N} \alpha_j\, \sigma(\mathbf{w}_j^\top \mathbf{x} + \theta_j), \qquad \mathbf{w}_j \in \mathbb{R}^n,\ \alpha_j, \theta_j \in \mathbb{R},\ N \in \mathbb{N},$$

are dense in $C(I_n)$: for every continuous f and $\varepsilon > 0$ some G satisfies $\sup_{\mathbf{x}\in I_n} |f(\mathbf{x}) - G(\mathbf{x})| < \varepsilon$.

The proof combines three ingredients: a Hahn–Banach consequence (Theorem 11.5), the Riesz representation theorem (Theorem 11.6), and the discriminatory property (Theorem 11.9).

3.1 The functional-analytic lemmas

Theorem 11.5: a consequence of Hahn–Banach

Let W be a linear, non-dense subspace of a normed linear space X. Then there is a bounded linear functional L on X with $L \neq 0$ on X and $L|_W = 0$.

Proof. The closure $\overline{W}$ is a closed subspace and $\overline{W} \neq X$, so pick $x_0 \notin \overline{W}$; then $d = \inf_{w\in\overline{W}} \|x_0 - w\| > 0$. On $\overline{W} \oplus \mathbb{R}x_0$ define $\ell(w + tx_0) = t$. It is linear, vanishes on W, $\ell(x_0) = 1$, and bounded: $|t|d \le |t|\|x_0 + w/t\| = \|w + tx_0\|$ for $t \neq 0$, so $\|\ell\| \le 1/d$. The Hahn–Banach theorem extends ℓ to X with the same norm. ∎

Theorem 11.6: Riesz representation theorem

Let $K \subset \mathbb{R}^n$ be compact. For every bounded linear functional F on $C(K)$ there is a unique finite signed (regular Borel) measure μ on K with

$$F(f) = \int_K f(\mathbf{x})\, d\mu(\mathbf{x}) \qquad \text{for all } f \in C(K).$$

(For positive functionals the measure is nonnegative.)

Theorem 11.7: an annihilating measure

Let W be a linear, non-dense subspace of $C(I_n)$. Then there is $\mu \in M(I_n)$, $\mu \neq 0$, with $\int_{I_n} h\, d\mu = 0$ for all $h \in W$.

Proof. Theorem 11.5 with $X = C(I_n)$ gives a bounded linear $L \neq 0$ with $L|_W = 0$. Riesz represents $L(f) = \int f\, d\mu$; $\mu \neq 0$ because $L \neq 0$; and for $h \in W$, $\int h\, d\mu = L(h) = 0$. ∎

3.2 Density for discriminatory functions

Theorem 11.8: density for discriminatory activations

If σ is continuous and discriminatory, the sums G above are dense in $C(I_n)$.

Proof. Since σ is continuous, $W = \{G\}$ is a linear subspace of $C(I_n)$ (sums and scalar multiples of such sums are again such sums). Suppose W is not dense. By Theorem 11.7 there is $\mu \neq 0$ with $\int h\, d\mu = 0$ for all $h \in W$, in particular

$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu(\mathbf{x}) = 0 \qquad \text{for all } \mathbf{w} \in \mathbb{R}^n,\ \theta \in \mathbb{R}.$$

Because σ is discriminatory, $\mu = 0$: a contradiction. ■

3.3 Continuous sigmoidals are discriminatory

Theorem 11.9: continuous sigmoidals are discriminatory

Every continuous sigmoidal function is discriminatory for all $\mu \in M(I_n)$.

Proof. Assume $\int \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu = 0$ for all $\mathbf{w}, \theta$. Fix $\mathbf{w}, \theta, \varphi$ and let $\sigma_\lambda(\mathbf{x}) = \sigma(\lambda(\mathbf{w}^\top \mathbf{x} + \theta) + \varphi)$. As $\lambda \to \infty$, $\sigma_\lambda \to 1$ on the open half-space $\Pi = \{\mathbf{w}^\top \mathbf{x} + \theta > 0\}$, $\to 0$ on $\{\mathbf{w}^\top \mathbf{x} + \theta < 0\}$ and $= \sigma(\varphi)$ on the hyperplane $H = \{\mathbf{w}^\top \mathbf{x} + \theta = 0\}$. The σ_λ are bounded by 1, so dominated convergence gives

$$0 = \int \sigma_\lambda\, d\mu \ \longrightarrow\ \mu(\Pi) + \sigma(\varphi)\mu(H) \qquad \text{for every } \varphi.$$

Letting $\varphi \to -\infty$ and $\varphi \to +\infty$ yields $\mu(\Pi) = 0$ and $\mu(H) = 0$ for all half-spaces and hyperplanes. Fix $\mathbf{w}$ and define $F(h) = \int h(\mathbf{w}^\top \mathbf{x})\, d\mu(\mathbf{x})$ for bounded measurable $h : \mathbb{R} \to \mathbb{R}$. F is linear and bounded, and $F(\mathbf{1}_{[\theta,\infty)}) = \mu(\Pi) + \mu(H) = 0$, $F(\mathbf{1}_{(\theta,\infty)}) = 0$, hence F vanishes on indicators of intervals, on simple functions, and by uniform approximation on all bounded measurable h. Taking $h = \cos$ and $h = \sin$ gives $\int e^{i\mathbf{w}^\top \mathbf{x}}\, d\mu(\mathbf{x}) = 0$ for every $\mathbf{w}$: the Fourier transform of μ vanishes, so $\mu = 0$. ■

Proof of Cybenko's theorem. Theorem 11.9 shows σ is discriminatory; Theorem 11.8 gives density. ■

Remark 11.10: what the theorem says and does not say

One hidden layer with enough neurons approximates any continuous function on a compact set uniformly (the cube can be replaced by any compact K after rescaling). The theorem is an existence result: it gives no bound on N, no algorithm to find the weights, and no statement about generalisation. Later work extends it to every continuous non-polynomial activation, including ReLU (Leshno et al. 1993). Intuition in 1D: a steep sigmoid is almost a step, a difference of two steps is a bump, and a sum of bumps approximates any continuous function.

4 Barron's Theorem: a Dimension-Free Rate

4.1 Fourier preliminaries

For $f \in L^1(\mathbb{R}^d)$ the Fourier transform is $\hat{f}(\boldsymbol{\xi}) = \int_{\mathbb{R}^d} f(\mathbf{x}) e^{-2\pi i \boldsymbol{\xi}^\top \mathbf{x}}\, d\mathbf{x}$. The Sobolev space $H^s(\mathbb{R}^d)$ consists of f with $\int |\hat{f}(\boldsymbol{\xi})|^2 (1 + |\boldsymbol{\xi}|^2)^s\, d\boldsymbol{\xi} < \infty$.

Definition 11.11: Barron class

For $C > 0$,

$$\Gamma_C = \Big\{ f \in L^1(\mathbb{R}^d) : \ \|\hat f\|_{L^1(\mathbb{R}^d)} < \infty, \ \int_{\mathbb{R}^d} |2\pi\boldsymbol{\xi}| \, |\hat f(\boldsymbol{\xi})| \, d\boldsymbol{\xi} < C \Big\}.$$

On a domain Ω one also uses the spectral Barron norm $\|f\|_{\mathcal{B}^s(\Omega)} = \inf_{f_e|_\Omega = f} \int_{\mathbb{R}^d} (1 + |\boldsymbol{\xi}|)^s |\hat f_e(\boldsymbol{\xi})| \, d\boldsymbol{\xi}$, the infimum over all extensions f_e of f beyond Ω.

Theorem 11.12: Barron 1993

Let σ be sigmoidal, $f \in \Gamma_C$ and $B_1^d = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| \le 1\}$. Then for every $c > 4C^2$ and every $N \in \mathbb{N}$ there is a network f_N with N neurons such that

$$\frac{1}{|B_1^d|} \int_{B_1^d} |f(\mathbf{x}) - f_N(\mathbf{x})|^2 \, d\mathbf{x} < \frac{c}{N}.$$

The rate $O(1/N)$ does not depend on the dimension d; for generic smooth functions (e.g. approximation by fixed bases) the number of terms needed typically grows exponentially in d. The price is the assumption $f \in \Gamma_C$, a condition on the Fourier spectrum.

4.2 Proof idea: Maurey's empirical method

Theorem 11.13: approximation of convex combinations

Let $\mathcal{H}$ be a Hilbert space and $\mathcal{G} \subset \mathcal{H}$ with $\|g\| \le b$ for all $g \in \mathcal{G}$. If f is in the closed convex hull of $\mathcal{G}$, then for every N there are $g_1, \dots, g_N \in \mathcal{G}$ with

$$\Big\| f - \frac{1}{N} \sum_{k=1}^{N} g_k \Big\|^2 \le \frac{b^2 - \|f\|^2}{N} + \varepsilon \quad (\text{any } \varepsilon > 0).$$

Proof. First let $f = \sum_m \lambda_m h_m$ be a finite convex combination. Draw $g_1, \dots, g_N$ iid with $P(g_k = h_m) = \lambda_m$; then $\mathbb{E} g_k = f$ and, by independence,

$$\mathbb{E}\Big\| f - \frac{1}{N}\sum_{k} g_k \Big\|^2 = \frac{1}{N} \mathbb{E}\|g_1 - f\|^2 = \frac{1}{N}\big(\mathbb{E}\|g_1\|^2 - \|f\|^2\big) \le \frac{b^2 - \|f\|^2}{N}.$$

A random variable is not always above its mean, so some realisation achieves the bound. For f in the closure, approximate it by a finite convex combination first. ∎

Barron writes $f(\mathbf{x}) - f(\mathbf{0})$, via Fourier inversion, as an average over frequencies of functions $\cos(2\pi\boldsymbol{\xi}^\top \mathbf{x} + \phi) - \cos\phi$, each of which is a limit of combinations of sigmoidal step functions $\sigma(\mathbf{w}^\top \mathbf{x} + \theta)$; the Barron norm bounds the total mass, so $f - f(\mathbf{0})$ lies in the closed convex hull of a set of functions bounded by $2C$ in L^2 of the normalised ball. Maurey's lemma with $b = 2C$ gives $4C^2/N$; any $c > 4C^2$ absorbs the ε.